

Mobile Development :

3 : Flutter for Mobile Development : Part 1

Introduction to Flutter : Widgets



Professor Imed Bouchrika

National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- **Section 1 : Coding Principles for Mobile**
 - Clean Code
 - Exceptions
 - Debugging & Step-by-step coding
- **Section 2 : Introduction to Flutter**
 - Hello World ! in Flutter
 - Flutter Project Structure
 - Useful Commands
 - Stateless & Stateful Widgets
 - Widgets for Creating a Flutter App.
 - Simple UI for a To-do Mobile App
- **Section 3 : Android OS**

Section 1

Coding Principles for Mobile Apps



Coding Principles for Mobile App Dev



- **Clean Code**

- Anyone can write code, but not all people can maintain code:
 - Maintain =
 - Understand the previous code.
 - Add extra features,
 - Fix bugs and remove errors.
- Statistics:
 - 1 minute spent on coding $\Rightarrow$ 9 minutes spent on reading code.

Coding Principles for Mobile App Dev



- **Clean Code**

- Naming (Variables , Functions, Classes..)
 - Must be expressive to what they are created for
 - Should be pronounceable
 - Variable : starts with a small letter
 - Function : starts with a small letter, better to start with a verb : (get, set, action, _check ...) + Avoid **void** functions
 - Class : Starts with a Capital letter.

Coding Principles for Mobile App Dev



- **Clean Code**

- Files and Structuring:
 - Indentation is a **must**
 - Avoid putting over 200 lines inside a single file.
 - Avoid **horizontal** and vertical scrolling on the screen
 - Files should be named based on what they contain.
 - Use Folders to group files which are related based on :
 - Domain (Sales , Accounting , ..)
 - Technical aspect (View, Widgets, Lib ...)

Coding Principles for Mobile App Dev

- **Clean Code**

- And lastly , many principles:

- *SOLID*

- *DRY*

- *KISS*

Coding Principles for Mobile App Dev

- Syntax of Dart : Exception Handling

```
try {  
    int number = int.parse(line);  
    var result = computeFactorial(number);  
    print("Result for $number ! is $result");  
} catch (e, stacktrace) {  
    print("Error ${e.toString()} stackTrace $stacktrace ");  
}
```


Coding Principles for Mobile App Dev



- **Syntax of Dart : Logging/Printing Messages to Debug**
 - Use simple print

```
print("Result for $number ! is $result");
```

Coding Principles for Mobile App Dev

- **Syntax of Dart : Logging/Printing Messages to Debug**

- Use the Logger Package

- flutter pub add logger

```
import 'package:logger/logger.dart';

final logger = Logger();
logger.d('Debug message');
logger.i('Info message');
logger.w('Warning message');
logger.e('Error message', error: error, stackTrace: stackTrace);
logger.f('Fatal/critical message');
```

Coding Principles for Mobile App Dev



- **Coding Step by Step**

- Don't write thousands of lines of code and later try to compile ?
- Write a few lines of code
 - Read well
 - Compile
 - Add print message to master what the code does.
 - It will give you a chance to clarify the algorithm well.

Section 2

Introduction to Flutter



Introduction to Flutter

- **What's Flutter**

- Flutter is an open-source software development framework which uses the Dart Programming language
- It is designed to build natively compiled applications for mobile (Both android and iOS), web, and desktop from a single codebase.
- Flutter was first released in 2017 by Google. It has gained significant popularity in the developer community due to its versatility and ease of use.



Introduction to Flutter

- **Why use Flutter instead of Kotlin/Java**

- Single Codebase for Multiplatforms
- High Performance due to compiling to native code
- Fast Development with hot reload where changes can be applied “instantly” to running apps without losing data or states.
- Support for Material design (Android theme) and Cupertino (iOS)
- Beautiful Interfaces and Rich Library of Widgets
- Cross-Platform Development where you can develop on various platforms for various platforms including desktop or even web and potentially backends.
- Backed and Supported by Google with Strong Community Support

Introduction to Flutter

- **Popular Apps made using Flutter**

- Google Classroom
- Google Pay
- eBay
- PUBG Mobile
- Toyota App
- Crédit Agricole
- Including apps by ByteDance, Tencent...



Hello World ! in Flutter

- Creating a Simple Hello World !

- Create a new Project

- Ctrl + Shift + P : Type **flutter** and choose : **New Flutter Project**



- Choose **Empty application**



Hello World

- main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Hello World App')),
        body: Center(
          child: Text('Hello World!'),
        ),
      ),
    );
  }
}
```

Hello World

```
import 'package:flutter/material.dart';
```

```
void main() {  
    runApp(const MainApp());  
}
```

```
class MainApp extends StatelessWidget {  
    const MainApp({super.key});  
  
    @override
```

- **main.dart**

- The main entry point to execute the App.
- **const** : used for Performance Optimization : Flutter creates const widget once at compile-time and reuses the same instance without rebuilding..

Hello World

- main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
```

- The screen being created is also a widget inheriting the stateless Widget Class
- Stateless Widget : **Immutable. Their properties cannot change once created = Built once.**

Hello World

- main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
```

- super.key is used internally by Flutter to identify widgets/components

- **build** : is the most important method to create the UI
- **MaterialApp** : Top-level widget that wraps your entire application providing essential app-wide functionality and configuration:
 - Navigation & Routing
 - Theming
 - Localization
 - Basic App Structure

```
@override
Widget build(BuildContext context) {
    return MaterialApp(
        home: Scaffold(
            appBar: AppBar(title: Text('Hello World App')),
            body: Center(
                child: Text('Hello World!'),
            ),
        ),
    );
}
```

- **MaterialApp** Properties:

- home - The default screen/widget to show
- debugShowCheckedModeBanner
- title - App name (shows in task switcher)
- theme - App-wide styling and colors
- darkTheme - Dark mode styling
- routes - Named routes for navigation

- **iOS vs. Android** : Can be used for Android, and even iOS. Alternatively , you can use : **CupertinoApp**

```
        appBar: AppBar(title: Text('Hello World App')),  
        body: Center(  
          child: Text('Hello World!'),  
        ),  
      ),  
    );  
  }  
}
```

- **Scaffold** widget provides the basic visual structure for **a screen in Material Design**.
- **Properties for the Scaffold:**
 - **appBar** - Top bar with title and actions
 - **body** - Main content area
 - **floatingActionButton** - Floating button (usually bottom-right)
 - **bottomNavigationBar** - Bottom navigation
 - **drawer** - Side menu that slides in
 - bottomSheet - Sheet that slides up from bottom
 - snackBar - Temporary messages at the bottom

```
home: Scaffold(  
  appBar: AppBar(title: Text('Hello World App')),  
  body: Center(  
    child: Text('Hello World!'),  
  ),  
),  
);  
  
}  
}
```

Hello World ! in Flutter

- **Everything is a Widget in a Flutter**

- The full App is a Widget
- The full screen is a Widget
- The button is a widget inside the screen widget.
- The text for the button label is a Widget
- The spacing between a button and another button, can be a widget.
- ...
- **The App is a tree of widgets.**

Hello World ! in Flutter

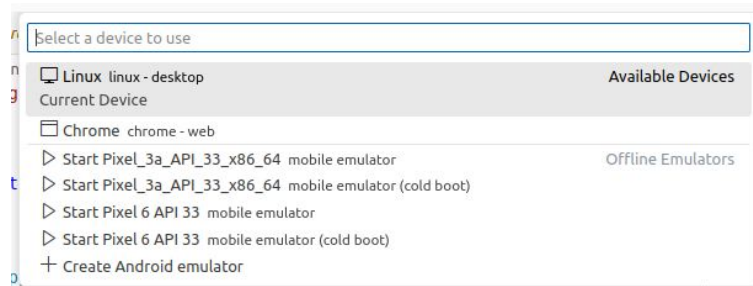
```
Widget(  
  VALUE_IF_NECESSARY  
  propertyOne:someData,  
  propertyOne:someData,  
  child: AnotherWidget...  
)
```

Hello World ! in Flutter

- To run

- Click on the device at the bottom right corner
- Choose an emulator or device
- Type in the terminal :

flutter run



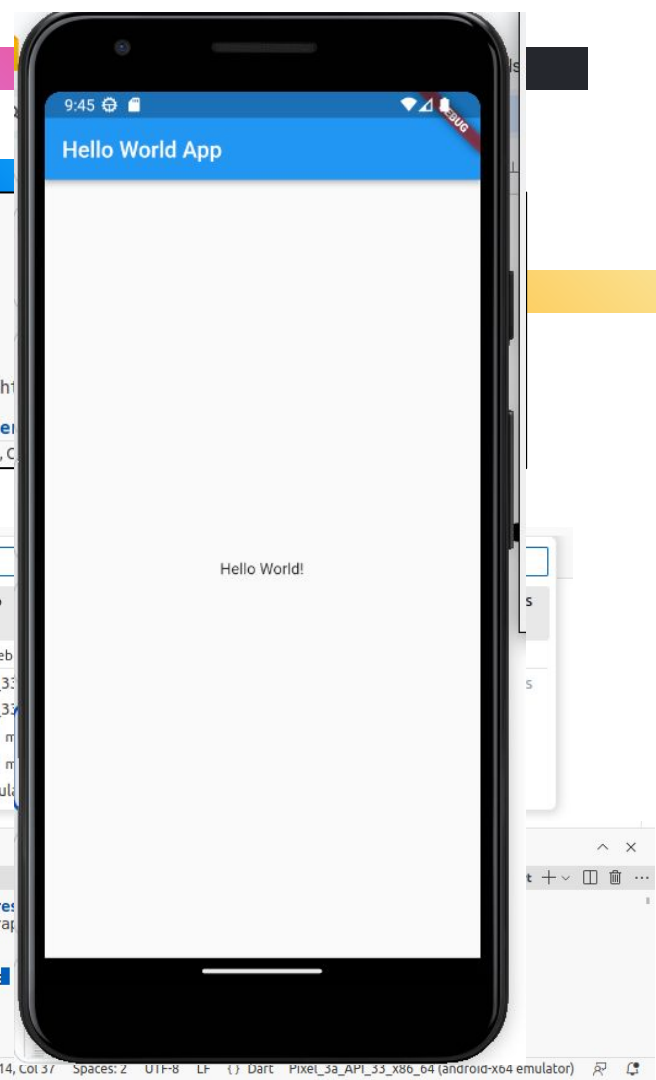
Hello World ! in Flutter

- To run

- Click on the device at the bottom right corner
- Choose an emulator or device
- Type in the terminal :

flutter run

```
TERMINAL
▼ TERMINAL
o imed@imed-Inspiron:~/Dropbox/Workspace/Teaching/ENSIA/MobileDev/MyLectures:
Using hardware rendering with device sdk gphone x86 64. If you notice graphi
"--enable-software-rendering".
Launching lib/main.dart on sdk gphone x86 64 in debug mode...
Running Gradle task 'assembleDebug'...
```

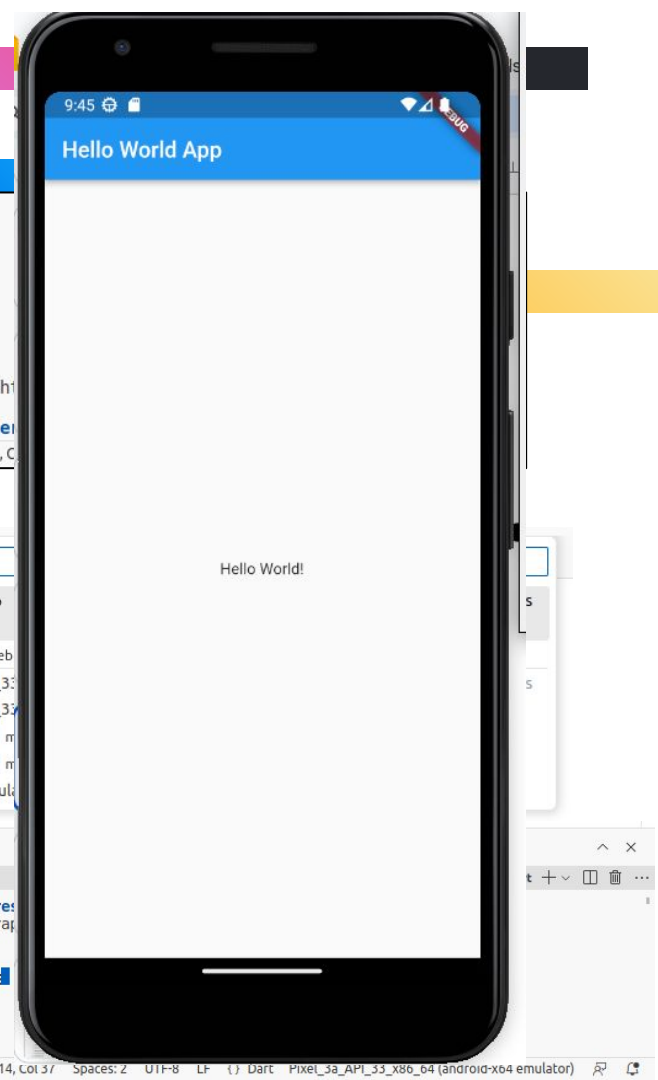


Hello World ! in Flutter

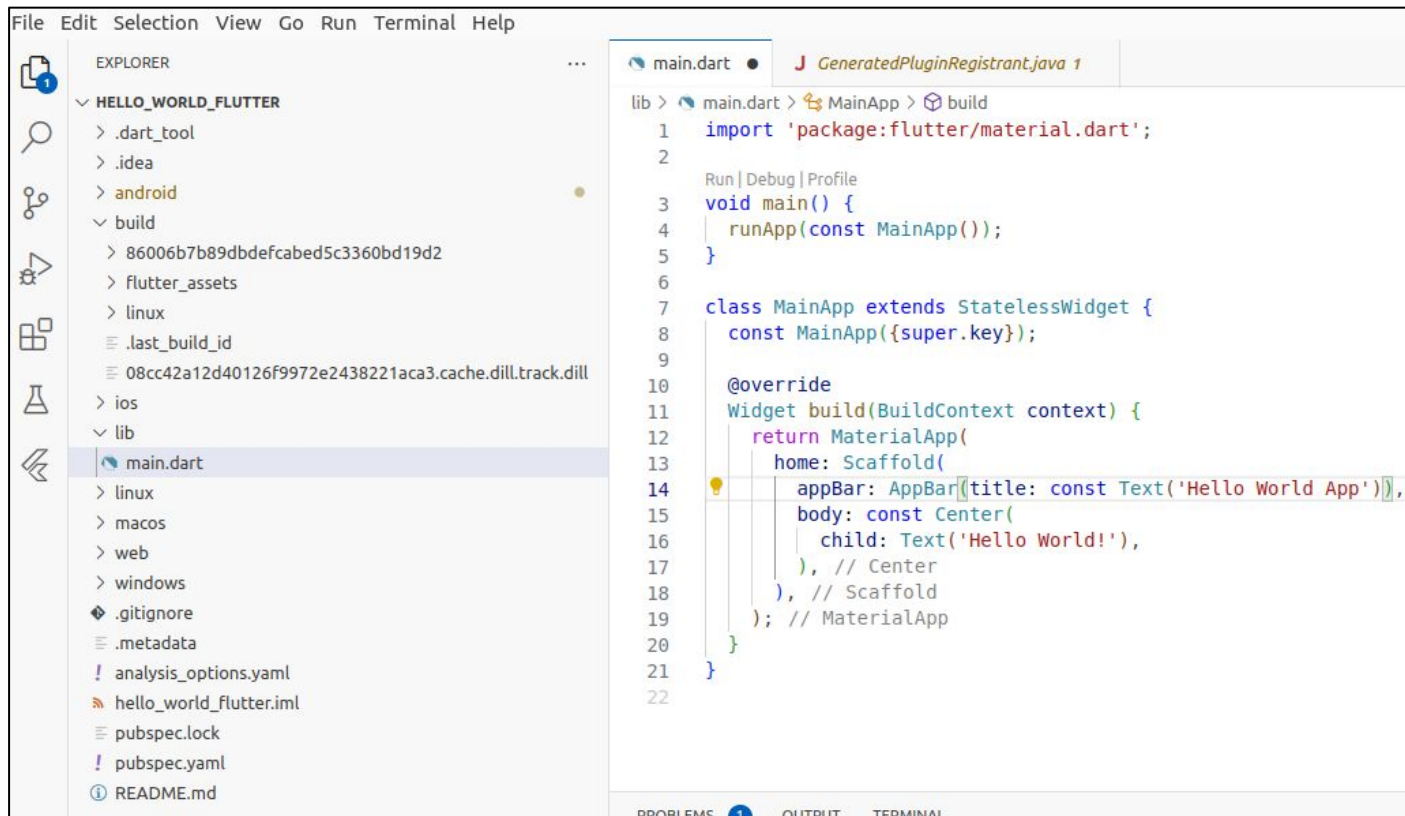
- **Hot reload**

- When you modify the code :

Inside the terminal, click the letter “**r**” to reload the App with the latest changes.

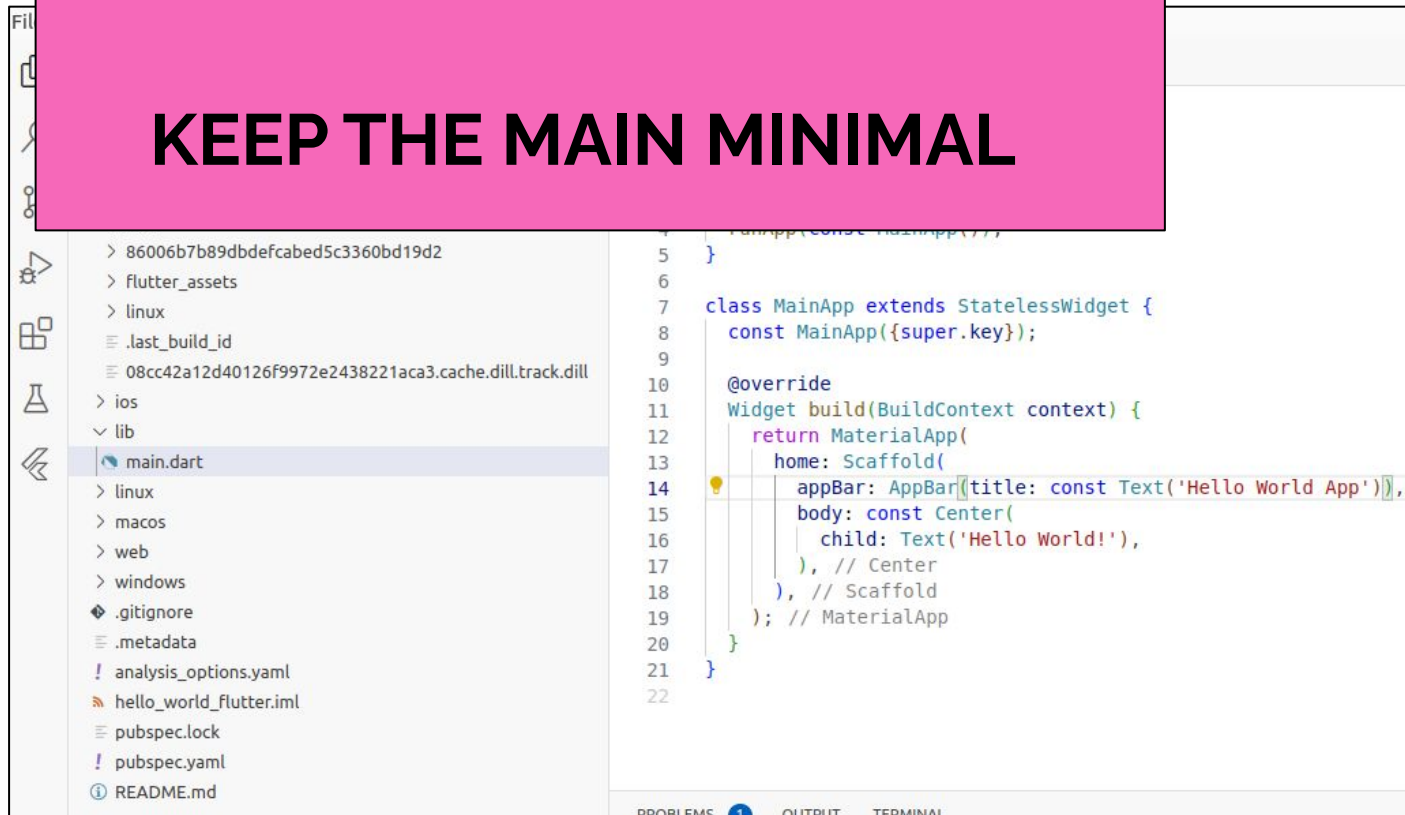


Flutter Project Structure



AVOID BUILDING WIDGET INSIDE THE MAIN FILE

KEEP THE MAIN MINIMAL



EXPLORER

HELLO_WORLD

- > .dart_tool
- > .idea
- > android
- > ios
- lib
 - sceens
 - home.dart
 - main.dart**
 - linux
 - macos
 - web
 - windows
 - .gitignore
 - .metadata
 - ! analysis_options.yaml
 - hello_world.iml
 - pubspec.lock
 - ! pubspec.yaml
 - ! README.md

home.dart X main.dart X

lib > main.dart > ...

```
1 import 'package:flutter/material.dart';
2 import 'package:hello_world/sceens/home.dart';
3
4 void main() {
5   runApp(const MainApp());
6 }
7
8 class MainApp extends StatelessWidget {
9   const MainApp({super.key});
10
11   @override
12   Widget build(BuildContext context) {
13     return const MaterialApp(home: HomeScreen());
14   }
15 }
16
```

main.dart

EXPLORER

HELLO... [Icons]

- > .dart_tool
- > .idea
- > android
- > ios
- ✓ lib
 - ✓ screens
 - home.dart
 - main.dart
 - > linux
 - > macos
 - > web
 - > windows
 - ◆ .gitignore
 - ≡ .metadata

home.dart × main.dart

lib > screens > home.dart > HomeScreen > build

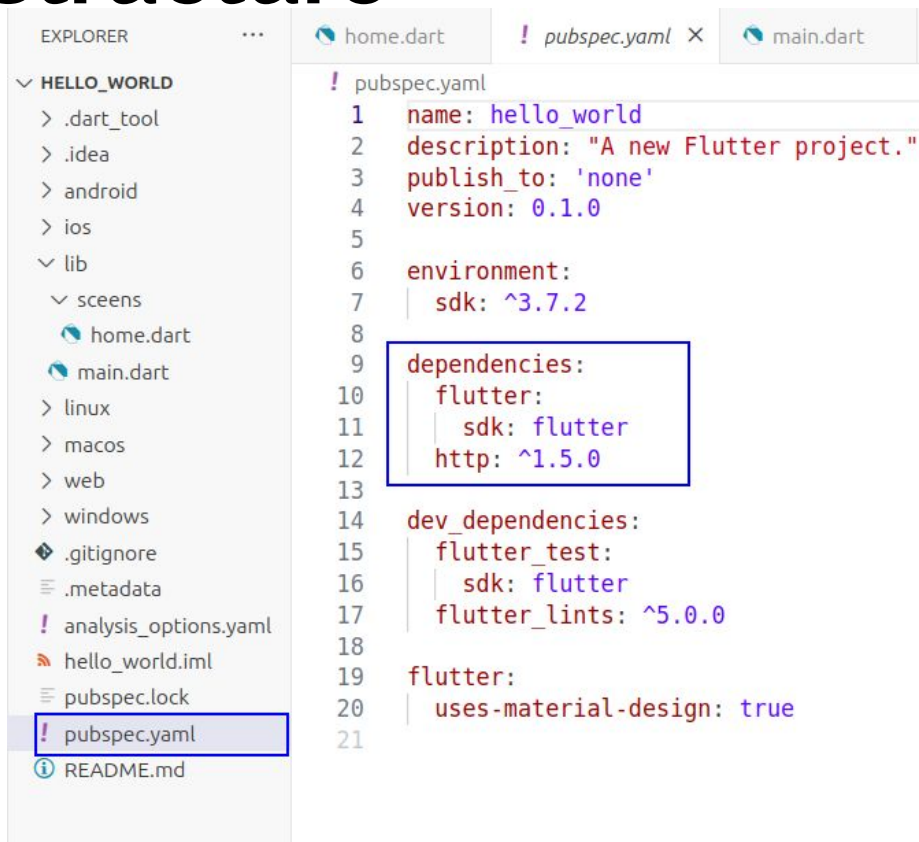
```
1 import 'package:flutter/material.dart';
2
3 class HomeScreen extends StatelessWidget {
4   const HomeScreen({super.key});
5
6   @override
7   Widget build(BuildContext context) {
8     return Scaffold(body: Center(child: Text('Hello World!')));
9   }
10 }
11
```

screens/home.dart

Flutter Project Structure

- **pubspec.yaml**

- is a configuration file that is used to define and manage:
 - Dependencies
 - Metadata
 - Settings
 - Assets (Images, Fonts...)



The screenshot shows an IDE interface. On the left, the 'EXPLORER' panel displays the project structure for 'HELLO_WORLD'. The files listed include .dart_tool, .idea, android, ios, lib, linux, macos, web, windows, .gitignore, .metadata, analysis_options.yaml, hello_world.iml, pubspec.lock, and **pubspec.yaml** (highlighted with a blue box). On the right, the 'pubspec.yaml' file is open, showing its contents. The file is numbered 1 through 21. The 'dependencies' section is highlighted with a blue box, showing the Flutter SDK dependency. The 'dev_dependencies' section shows dependencies for flutter_test and flutter_lints. The 'flutter' section at the bottom shows 'uses-material-design: true'.

```
! pubspec.yaml
1  name: hello_world
2  description: "A new Flutter project."
3  publish_to: 'none'
4  version: 0.1.0
5
6  environment:
7    sdk: ^3.7.2
8
9  dependencies:
10    flutter:
11      sdk: flutter
12    http: ^1.5.0
13
14  dev_dependencies:
15    flutter_test:
16      sdk: flutter
17    flutter_lints: ^5.0.0
18
19  flutter:
20    uses-material-design: true
21
```



Flutter Project Structure

- **Where to find flutter libraries**

- Visit <https://pub.dev/> and search for whatever feature you want to add :

Flutter Project Structure

- Where to find flutter libraries

The screenshot shows the pub.dev website with a search bar containing 'datepicker'. The search results are filtered by 'Platforms' (Android, iOS, Linux, macOS, Web, Windows) and 'SDKs'. The results list three packages: 'calendar_date_picker2', 'scroll_date_picker', and 'syncfusion_flutter_datepicker'. The 'syncfusion_flutter_datepicker' package is highlighted with a red box around its statistics: 1266 Likes, 130 Pub Points, and 99% Popularity. The package description for 'syncfusion_flutter_datepicker' states: 'The Flutter Date Range Picker widget allows users to easily select dates or a range of dates. It has four built-in views that allow quick navigation to the desired date.'

pub.dev

Sign in Help

datepicker

Platforms

- ☐ Android
- ☐ iOS
- ☐ Linux
- ☐ macOS
- ☐ Web
- ☐ Windows

SDKs

License

Advanced

RESULTS 657 packages

SORT BY SEARCH RELEVANCE

calendar_date_picker2

256 LIKES 140 PUB POINTS 98% POPULARITY

A lightweight and customizable calendar picker based on Flutter CalendarDatePicker, with support for single date picker, range picker and multi picker.

v 0.5.3 (2 months ago) Apache-2.0 Dart 3 compatible

SDK FLUTTER PLATFORM ANDROID IOS LINUX MACOS WEB WINDOWS

scroll_date_picker

74 LIKES 140 PUB POINTS 95% POPULARITY

A customizable and easy-to-use date picker library for Flutter. Compatible with Android & iOS & Web.

v 3.7.3 (4 months ago) BSD-3-Clause Dart 3 compatible

SDK FLUTTER PLATFORM ANDROID IOS

syncfusion_flutter_datepicker

1266 LIKES 130 PUB POINTS 99% POPULARITY

The Flutter Date Range Picker widget allows users to easily select dates or a range of dates. It has four built-in views that allow quick navigation to the desired date.

Flutter Project Structure

- **Version of the libraries:**

dependencies:

flutter:

 sdk: flutter

cupertino_icons: ^1.0.5

sqlite: ^2.2.8+2

path:

odoo_rpc: ^0.5.1

convert: ^3.1.1

provider: ^6.0.5

flutter_launcher_icons: ^0.13.1

sqlite_common_ffi: any

sqlite_common_ffi_web: any

flutter_barcode_scanner: ^2.0.0

sqlite3_flutter_libs: any

image_picker: ^0.8.7+5

flutterntoast: ^8.2.2

syncfusion_flutter_datepicker 23.1.43

Published 17 hours ago • syncfusion.com Dart 3 compatible

SDK FLUTTER PLATFORM ANDROID IOS LINUX MACOS WEB WINDOWS

1.2K

Readme Changelog Example Installing Versions Scores



The ^ sign (Caret Syntax) = Any version guaranteed to be backwards compatible with the specified version.

Useful Commands

- To run the project
 - **flutter run**
- To add a package.
 - **flutter pub add packageName**
- To run unit tests
 - **flutter test**
- Build APK
 - **Flutter build apk**
- Check flutter installation
 - **flutter doctor**
- Clean and remove build files.
 - **flutter clean**
- To run in release mode
 - **flutter run --release**
- Upgrade packages
 - **flutter pub upgrade**

Stateless & Stateful Widgets



- **Stateless Widgets**

- Immutable. Their properties cannot change once created.
- The build method is called only once.
- Used for UI elements that don't change, like an icon or a label.

Stateless & Stateful Widgets

- Stateless Widgets

```
import 'package:flutter/material.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(body: Center(child: Text('Hello World!')));
  }
}
```

Stateless & Stateful Widgets

- Stateless Widgets

```
import 'package:flutter/material.dart';
import 'package:hello_world/sceens/home.dart';
void main() {
  runApp(const MainApp());
}
class MainApp extends StatelessWidget {
  const MainApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(home: HomeScreen());
  }
}
```


Stateless & Stateful Widgets



- **Stateful Widgets**

- Mutable - can change over time
- Has internal state (**created as a separate class**)
- When the state changes, the widget is rebuilt by calling **setState()**
- Used for UI that needs to be redrawn, like a checkbox or a text field.

Stateless & Stateful Widgets

- Stateful Widgets

```
import 'package:flutter/material.dart';
import 'package:stateful_exampleeeeeeeeeee/screens/home.dart';
void main() {
  runApp(const MainApp());
}
class MainApp extends StatelessWidget {
  const MainApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(home: HomeScreen());
  }
}
```

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  var _counter = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: GestureDetector(
          onTap: () {
            _counter++;
            setState(() {});
          },
          child: Text('Hello World! $_counter '),
        ),
      ),
    );
  }
}
```

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  var _counter = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: GestureDetector(
          onTap: () {
            _counter++;
            setState(() {});
          },
          child: Text('Hello World! $_counter '),
        ),
      ),
    );
  }
}
```

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  var _counter = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: GestureDetector(
          onTap: () {
            _counter++;
            setState(() {});
          },
          child: Text('Hello World! $_counter '),
        ),
      ),
    );
  }
}
```

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  var _counter = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: GestureDetector(
          onTap: () {
            _counter++;
            setState(() {});
          },
          child: Text('Hello World! $_counter '),
        ),
      ),
    );
  }
}
```

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  var _counter = 0;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: GestureDetector(
          onTap: () {
            _counter++;
            setState(() {});
          },
          child: Text('Hello World! $_counter '),
        ),
      ),
    );
  }
}
```

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  var _counter = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: GestureDetector(
          onTap: () {
            _counter++;
            setState(() {});
          },
          child: Text('Hello World! $_counter '),
        ),
      ),
    );
  }
}
```


...

```
class _HomeScreenState extends State<HomeScreen> {  
  var _counter = 0;
```

```
  @override  
  void initState() {  
    super.initState();  
    _counter = 6;  
    //or initialize other settings...  
  }
```

```
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Center(  
        child: GestureDetector(  
          onTap: () {  
            _counter++;  
            setState(() {});  
          },  
          child: Text('Hello World! $_counter '),  
        ),  
      ),  
    );  
  }  
}
```

```

class HomeScreen extends StatefulWidget {
  Var some_variable=1
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {

```

```

  @override
  Widget build(BuildContext context)

```

```

    var data=widget.some_variable

```

```

    return Scaffold(
      body: Center(
        child: GestureDetector(
          onTap: () {
            _counter++;
            setState(() {});
          },
          child: Text('Hello World! $_counter / /',

```

Within the state class, to access the attributes/variables within the stateful widget, use the special variable : widget

Widgets for Creating a Flutter App

- Basic Widgets: Text

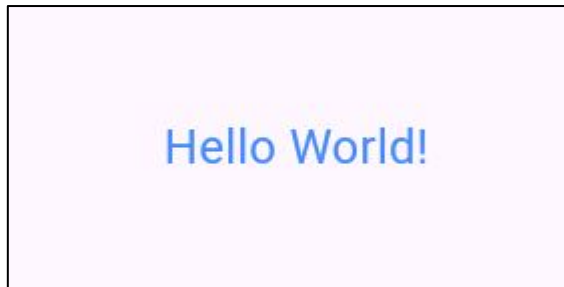
```
import 'package:flutter/material.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Text('Hello World!'),
    );
  }
}
```

Widgets for Creating Flutter App

- Basic Widgets: Text



```
import 'package:flutter/material.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Text(
          'Hello World!',
          textAlign: TextAlign.center,
          style: TextStyle(
            fontSize: 24,
            color: Colors.blueAccent),
        ),
      ),
    );
  }
}
```

Widgets for C Flutter A

- Basic Widgets: Text



Hello World!

```
TextStyle(  
  fontSize: 20,  
  fontWeight: FontWeight.bold,  
  fontStyle: FontStyle.italic,  
  color: Colors.blue,  
  backgroundColor: Colors.yellow,  
  letterSpacing: 2.0,  
  wordSpacing: 5.0,  
  decoration: TextDecoration.underline,  
  decorationColor: Colors.red,  
  decorationStyle: TextDecorationStyle.wavy,  
  height: 1.5,  
  shadows: [  
    Shadow(  
      color: Colors.black,  
      offset: Offset(2, 2),  
      blurRadius: 3),  
  ],  
)
```

Widgets for O Flutter A

- Basic Widgets: Text



```
TextStyle(  
  fontSize: 20,  
  fontWeight: FontWeight.bold,  
  fontStyle: FontStyle.italic,  
  color: Colors.blue,  
  backgroundColor: Colors.yellow,  
  letterSpacing: 2.0,  
  wordSpacing: 5.0,  
  decoration: TextDecoration.underline,  
  decorationColor: Colors.red,  
  decorationStyle: TextDecorationStyle.wavy,  
  height: 1.5,  
  shadows: [  
    Shadow(  
      color: Colors.black,  
      offset: Offset(2, 2),  
      blurRadius: 4,  
    ),  
  ],  
)
```

DRY : DON'T REPEAT YOURSELF

```
import 'package:flutter/material.dart';

const backColor = Color(0xffeff9ff);
const blackColor = Color(0xff000000);
const whiteColor = Color(0xffffffff);
const white1Color = Color(0xfff2f2f2);
const textColor = Color(0xff1e2123);
const text1Color = Color(0xff817f8c);
const brownColor = Color(0xff999999);
...
```

```
import 'package:flutter/material.dart';
import 'colors.dart';
```

```
const startTextStyle = TextStyle(
  fontSize: 25.0,
  color: textColor,
  fontFamily: "Satoshi",
  fontWeight: FontWeight.w700,
  fontStyle: FontStyle.normal,
);
const buttonTextStyle = TextStyle(
  fontSize: 18.0,
  color: whiteColor,
  fontFamily: "Satoshi",
  fontWeight: FontWeight.w600,
  fontStyle: FontStyle.normal,
);
...
```

ing a

The diagram illustrates the flow of code dependencies. A pink box highlights the word 'ing a' in the text 'ing a'. A black arrow points from this text to the 'colors.dart' file in a file explorer. Another black arrow points from the 'colors.dart' file to the 'style: startTextStyle,' line in the widget build method. A large blue arrow points from the 'startTextStyle' definition in the second code block to the 'style: startTextStyle,' line in the widget build method.

```
> ios
  ▾ lib
    ▾ commons
      colors.dart
      images.dart
      styles.dart
    ▾ screens
      newCourse.dart
      registerScreen.dart
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: Text(
        'Hello World!',
        textAlign: TextAlign.center,
        style: startTextStyle,
      ),
    ),
  );
}
```

Widgets for Creating a Flutter App



- **Basic Widgets: Container**
 - Can be considered as a DIV in html
 - Major properties for a container:
 - **width**
 - **height**
 - **margin**
 - **padding**
 - **alignement**
 - **decoration**

Widgets for Creating a Flutter App

- **Basic Widgets: Container**

- Can be considered as a DIV in html
- Major properties for a container:

- **width**
- **height**
- **margin**
- **padding**
- **alignement**
- **decoration**

For full width across the screen use :

- width : **double.infinity**
- width: **MediaQuery.of(context).size.width**

Widgets for Creating a Flutter App

- **Basic Widgets: Container : decoration : BoxDecoration**
 - **Background**
 - **color** - Background color of the box
 - **image** - Background image (DecorationImage)
 - **gradient** - Gradient background (LinearGradient,...)
 - **Border & Shape**
 - **border** - Border around the box (Border.all, Border.symmetric, etc.)
 - **borderRadius** - Rounded corners (BorderRadius)
 - **shape** - Shape of the box (BoxShape.rectangle or BoxShape.circle)
 - **Shadow & Effects**
 - **boxShadow** - List of shadows cast by the box (List<BoxShadow>)

```
Container(
```

```
  width: 200,
```

```
  height: 100,
```

```
  margin: EdgeInsets.all(16),
```

```
  padding: EdgeInsets.symmetric(horizontal: 20, vertical: 10),
```

```
  alignment: Alignment.center,
```

```
  decoration: BoxDecoration(
```

```
    color: Colors.blue,
```

```
    borderRadius: BorderRadius.circular(12),
```

```
    border: Border.all(color: Colors.black, width: 2),
```

```
    boxShadow: [
```

```
      BoxShadow(color: Colors.grey, offset: Offset(2, 2), blurRadius: 5),
```

```
    ],
```

```
    gradient: LinearGradient(colors: [Colors.blue, Colors.purple]),
```

```
  ),
```

```
  child: Text('Hello'),
```

```
)
```

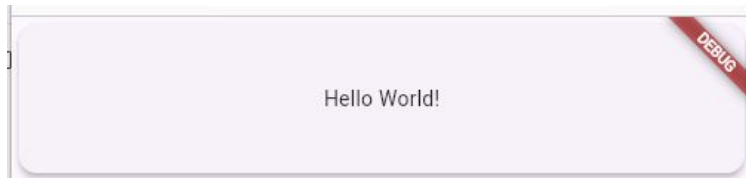


Widgets for C Flutter A

- Basic Widgets: Card



```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Card(
      margin: EdgeInsets.all(5),
      elevation: 4,
      child: Container(
        width: double.infinity,
        alignment: Alignment.center,
        height: 100,
        child: Text('Hello World!'),
      ),
    ),
  );
}
```



Widgets for O Flutter A

- Basic Widgets: ListTile

```
Center(  
  child: Card(  
    elevation: 4,  
    child: ListTile(  
      leading: Icon(Icons.person),  
      title: Text('Mobile Dev'),  
      subtitle: Text('3rd Year Students'),  
      trailing: Icon(Icons.arrow_forward),  
      onTap: () {  
        // for handling events  
      },  
    ),  
  ),  
)
```



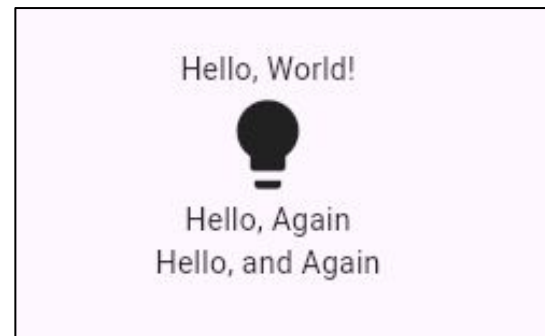
Mobile Dev
3rd Year Students



Widgets for Creating a Flutter App

- **Layout Widgets: Column & Row**

- Arranges a list of child widgets in a vertical or Horizontal way.
 - Column : Vertical
 - Row : Horizontal
- Main property is :**children**
 - **Takes a List []**



Widgets for Flutter

- Layout Widgets:

```
Scaffold(  
  body: Container(  
    width: double.infinity,  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      crossAxisAlignment: CrossAxisAlignment.center,  
      children: const [  
        Text('Hello, World!'),  
        Icon(Icons.lightbulb, size: 50),  
        Text('Hello, Again'),  
        Text('Hello, and Again'),  
      ],  
    ),  
  ),  
)
```

Hello, World!



Hello, Again
Hello, and Again

Widgets for Flutter

- Layout Widgets:

```
Scaffold(  
  body: Container(  
    width: double.infinity,  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      crossAxisAlignment: CrossAxisAlignment.center,  
      children: const [  
        Text('Hello, World!'),  
        Icon(Icons.lightbulb, size: 50),  
        Text('Hello, Again'),  
        Text('Hello, and Again'),  
      ],  
    ),  
  ),  
)
```

Hello, World!  Hello, AgainHello, and Again

Widget Control-Flow ? within Flutter Widget ?

- Layout Widgets:

```
mainAxisAlignment: MainAxisAlignment.center,  
crossAxisAlignment: CrossAxisAlignment.center,  
children: [  
  
    for(int i=0;i<6;i++)  
        Text('Hello, World! $i '),  
  
    Icon(Icons.lightbulb, size: 50),  
    Text('Hello, Again'),  
    Text('Hello, and Again'),  
],  
,  
,  
,
```

Widgets for Creating a Flutter App

```
children: const [  
  Text('Hello, World!'),  
  SizedBox(height: 20),  
  Icon(Icons.lightbulb, size: 50),  
  Text('Hello, Again'),  
  Text('Hello, and Again'),  
],
```

- **Layout Widgets: Spacing within Rows/Columns**
 - **SizedBox** : can be used :
 - Adds fixed space between widgets:
 - Or even encapsulates widgets within a fixed sized box
 - Attributes
 - width OR height OR both

Widgets for Creating a Flutter App

```
children: const [  
  Text('Hello, World!'),  
  Spacer(),  
  Icon(Icons.lightbulb, size: 50),  
  Text('Hello, Again'),  
  Text('Hello, and Again'),  
],
```

- **Layout Widgets: Spacing within Rows/Columns**
 - **Spacer** : can be used :
 - To fill spaces between widgets
 - Works only inside a Row/Column
 - The height OR width of the containing parent must have a defined bounded (defined width for row or height for column)

Widgets for Creating a Flutter App

- **Layout Widgets: Spacing within Rows/Columns**
 - **Expanded** : can be used :
 - To force its **child** to fill all the available space.
 - The parent **ust have a bounded size**.

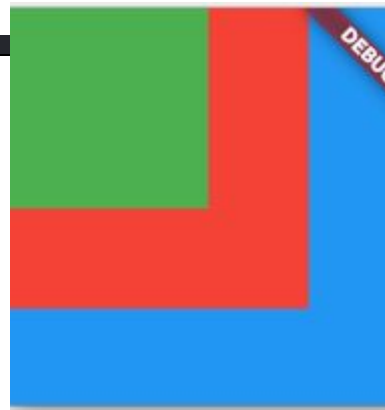
```
Row(  
  children: [  
    Container(width: 100, color: Colors.red),  
    Expanded(child: Container(color: Colors.blue)),  
    Container(width: 100, color: Colors.green),  
  ],  
)
```



Widgets for Creating Flutter App

- Layout Widgets: Stack
 - Allow to **overlay** widgets on top of each other.

```
Stack(  
  children: [  
    Container(  
      width: 200,  
      height: 200,  
      color: Colors.blue,  
    ), // Bottom layer  
    Container(  
      width: 150,  
      height: 150,  
      color: Colors.red,  
    ), // Middle layer  
    Container(  
      width: 100,  
      height: 100,  
      color: Colors.green,  
    ), // Top layer  
  ],  
)
```



Widgets for Creating a Flutter App

- **Layout Widgets: Stack**

- Positioned widget can be used
For absolute placement by setting
The attributes :

left, top, bottom or right

```
Stack(  
  children: [  
    Container(width: 300,  
              height: 300, color: Colors.grey),  
    Positioned(  
      top: 20,  
      left: 20,  
      child: Icon(Icons.star, size: 50)),  
  
    Positioned(  
      bottom: 20,  
      right: 20,  
      child: Icon(Icons.favorite, size: 50),  
    ),  
  ],  
)
```

Widgets for Creating a Flutter App



- **Layout Widgets: Stack**

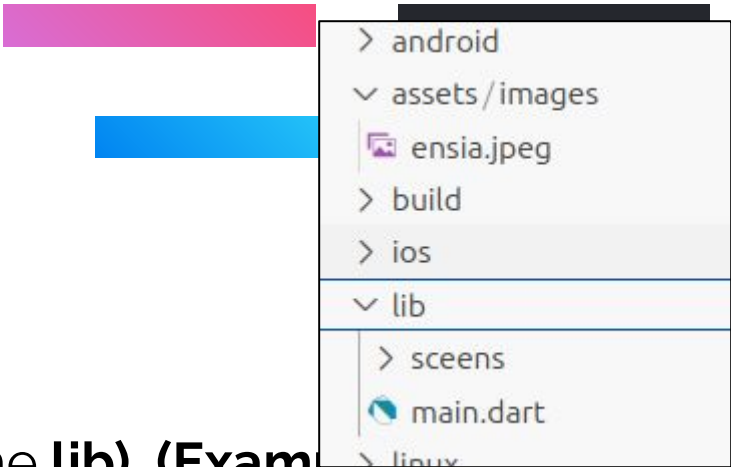
- You may even use for positioning Child widget inside a Stack using :

- `Positioned.Fill ( child : )` : To fill the child widget on all over the Stack.
- `Align`

Widgets for Creating a Flutter App

- **Image Widgets:**

1. Create an asset folder (not inside the **lib**) (**Example: `assets/images`**) and add your image there.
2. Update the pubspec file via adding the **assets**
3. You may need to run :
flutter pub get



```
> android
✓ assets/images
  ensia.jpeg
  > build
  > ios
  ✓ lib
    > screens
    main.dart
  > linux
```

```
flutter:
  uses-material-design: true
  assets:
    - assets/images/
```


Widgets for Creating a Flutter App

- Image Widgets:

- Flutter Widgets :

- `Image.asset`
 - `Image.network`

```
return Scaffold(  
  body: Container(  
    width: double.infinity,  
    child: Center(  
      child: Image.asset(  
        'assets/images/ensia.jpeg',  
        width: 400,  
        height: 200,  
      ),  
    ),  
  ),  
)
```

Widgets for Creating Flutter Apps

- **Data Widgets: ListView**
 - Scrollable widget
 - Lazy loading
 - Most cases, we use `ListView.builder`
 - Simple `ListView` is like a column

```
class HomeScreen extends StatelessWidget {
  HomeScreen({super.key});

  List<Map<String, dynamic>> data = [
    {'title': 'Software Engineering', 'done': true},
    {'title': 'Mobile Development', 'done': false},
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: ListView.builder(
        itemCount: data.length,
        itemBuilder: (context, index) {
          return Card(
            child: Container(
              height: 40,
              alignment: Alignment.centerLeft,
              child: Text('${data[index]['title']}'),
            ),
          );
        },
      ),
    );
  }
}
```

Widgets for Creating a Flutter App

- Forms & User Input Widgets :
 - ElevatedButton



```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: ElevatedButton(
        onPressed: () {
          print('Button pressed!');
        },
        child: Text('Click Me'),
      ),
    ),
  );
}
```

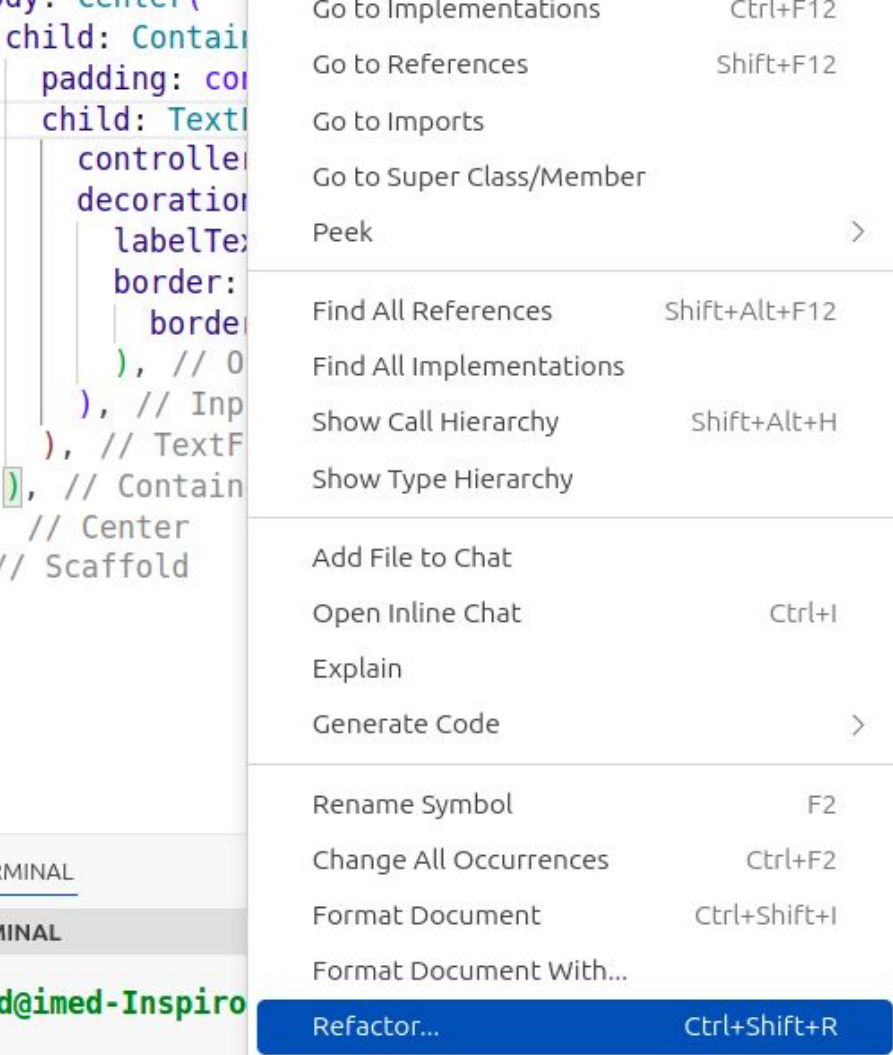
Widgets for C Flutter A

- Forms & User Input Widgets
 - TextField



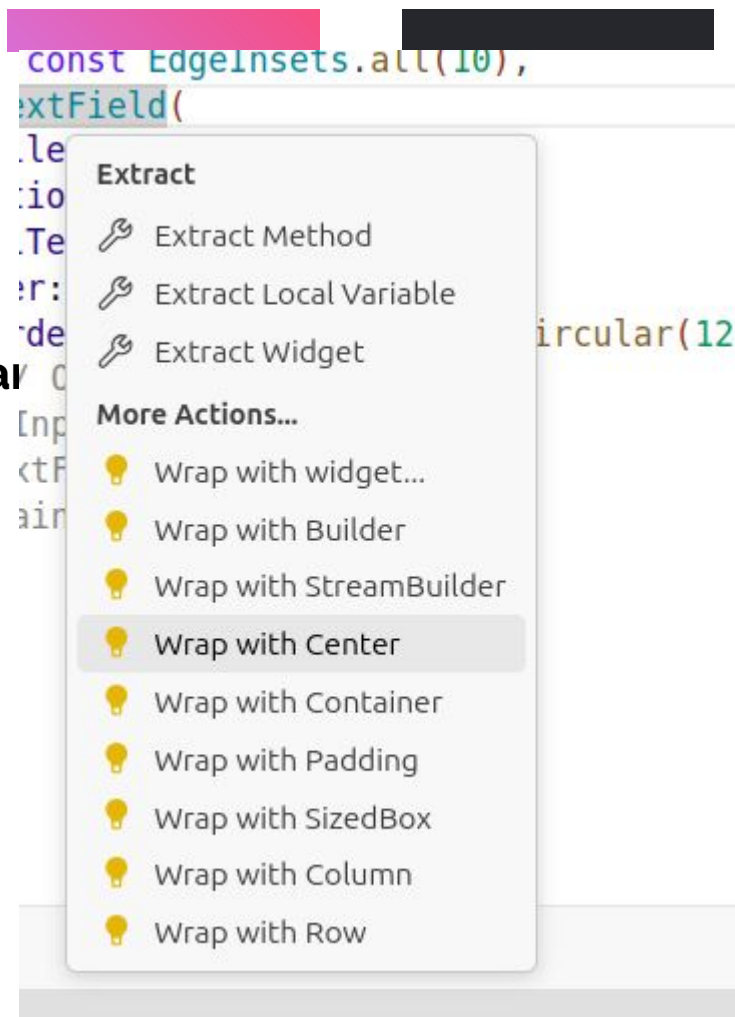
```
final TextEditingController _controller = TextEditingController();

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: Container(
        padding: const EdgeInsets.all(10),
        child: TextField(
          controller: _controller,
          decoration: InputDecoration(
            labelText: 'Email',
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
        ),
      ),
    ),
  );
}
```



ng a

ts with ar

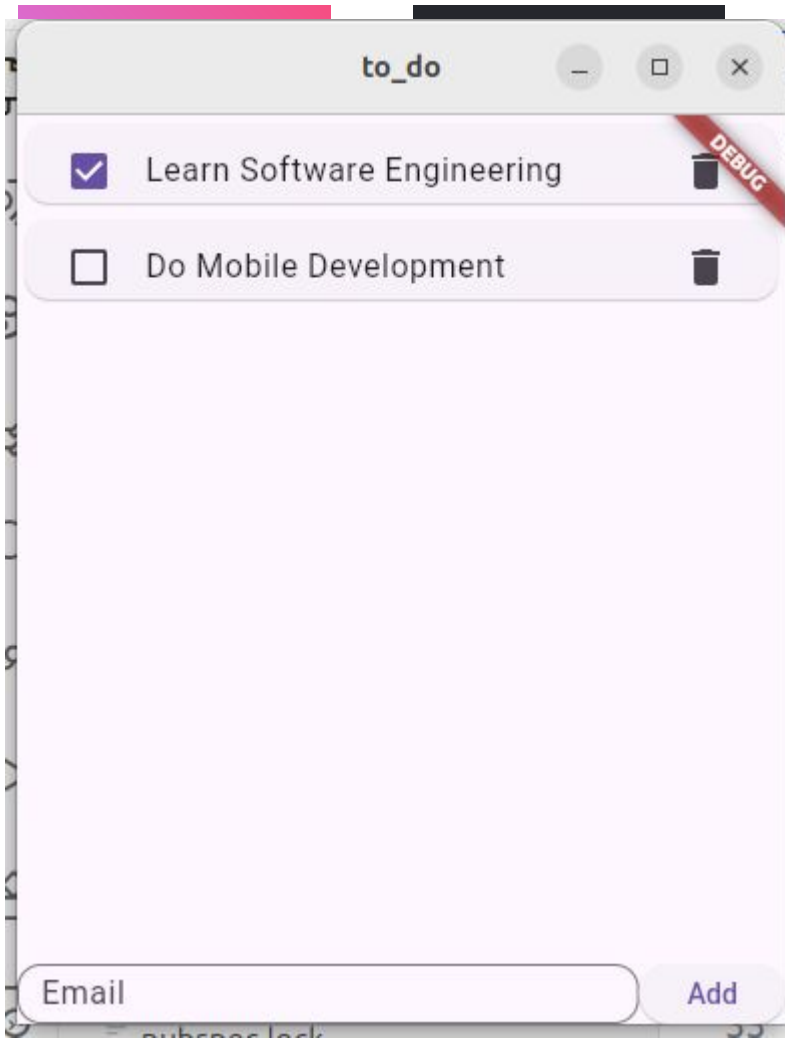
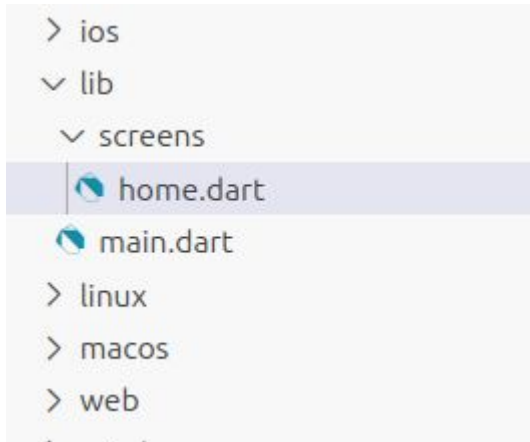


Widgets for Creating a Flutter App

- **Scaffold** : appBar, body, floatingActionButton, drawer, bottomNavigationBar
- **AppBar** : title , leading, elevation
- **Container** : child, padding, margin, height, width, decoration alignment
- **Column** : children, mainAxisAlignment, crossAxisAlignment
- **Row** : children, mainAxisAlignment, crossAxisAlignment
- **Stack** : alignment, children,
- **SizedBox** : child ,height, width ,
- **Card** : child, color, elevation, margin, clipBehavior, shape
- **Expanded** : child, flex
- **Spacer** : flex
- **Padding** : padding --> EdgeInsets.all|only|fromLTRB|symmetric
- **Center** : child
- **Align** : child, alignment
- **ListView.builder** : scrollDirection, physics, itemCount, itemBuilder(context, index)
- **GridView.builder** : padding, physics, itemCount, itemBuilder(context, index)
- **Text** : style, textAlign
- **TextFormField** : onChanged, controller, decoration, validator
- **ElevatedButton** : style, onPressed, child
- **IconButton** : icon, onPressed
- **Icon** : color, size,
- **Slider** : value, max, divisions, onChanged, label
- **Switch** : value, onChanged, thumbIcon
- **ListTile** : leading, title, subtitle, trailing, tileColor, onTap
- **Image.asset** : imagePath,
- **TextStyle** : fontSize, color, fontWeighth
- **BoxDecoration** : image, shape, border, borderRadius, color
- **Opacity** : opacity, child
- **RoundedRectangleBorder** : borderRadius, side
- **Position** : left, bottom, right, top
- **BoxShadow** : color, offset, blurStyle

Creating the UI for a simple To-do App

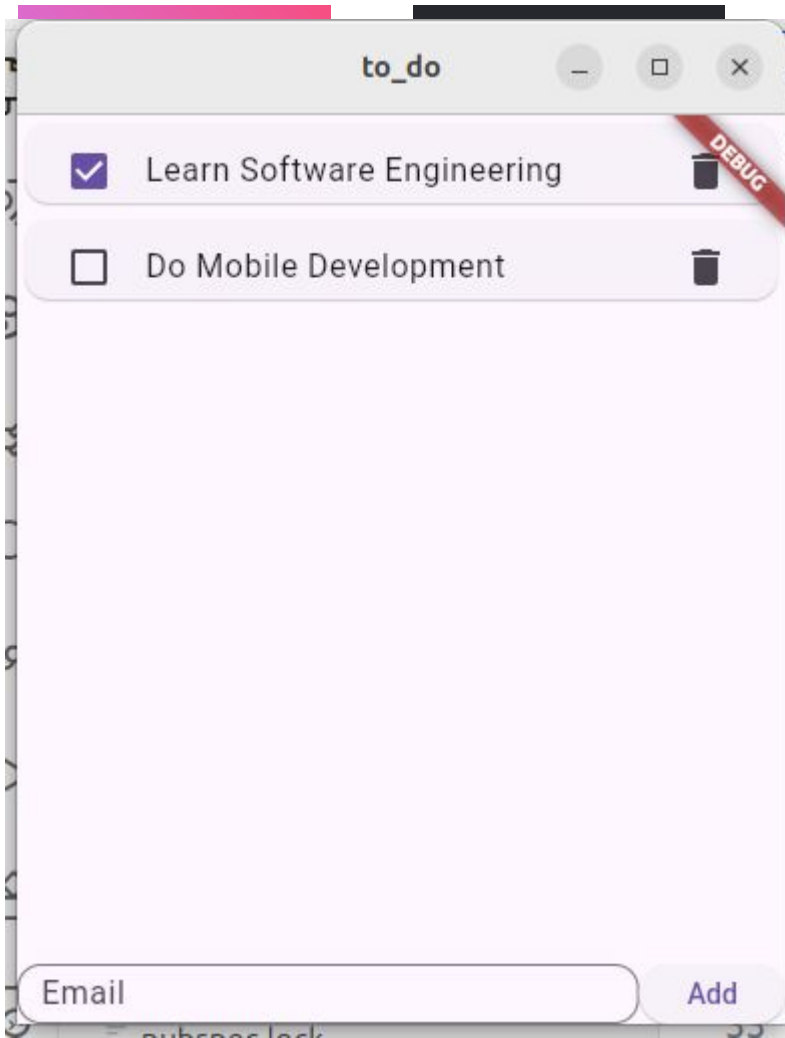
- **Stateful Widget**



Creating the UI for a simple To-do App

- **Stateful Widget**

- Scaffold:
 - Column:
 - Expanded:
 - ListView
 - Card
 - Container
 - ListTile
 - Text
 - SizedBox
 - Row
 - Expanded
 - TextField
 - ElevatedButton




```

import 'package:flutter/material.dart';
class TodoHomeScreen extends StatefulWidget {
  const TodoHomeScreen({super.key});
  @override
  State<TodoHomeScreen> createState() => _TodoHomeScreenState();
}
class _TodoHomeScreenState extends State<TodoHomeScreen> {
  List<Map<String, dynamic>> data = [
    {'title': 'Learn Software Engineering', 'done': true},
    {'title': 'Do Mobile Development', 'done': false},
  ];
  final TextEditingController _controller = TextEditingController();
  @override

  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: [
          Expanded(
            child: ListView.builder(
              itemCount: data.length,
              itemBuilder: (context, index) {
                return getItemTodo(data[index]);
              },
            ),
          ),
        ],
      ),
    ),
  ),

```

```

    SizedBox(
      height: 30,
      child: Row(
        children: [
          Expanded(
            child: TextField(
              controller: _controller,
              decoration: InputDecoration(
                labelText: 'Email',
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(12),
                ),
              ),
            ),
          ),
          ElevatedButton(onPressed: () {}, child: Text('Add')),
        ],
      ),
    ),
  ],
),
);
}
}

```

```
Widget getItemTodo(Map<String, dynamic> item) {  
  return Card(  
    child: Container(  
      height: 40,  
      child: ListTile(  
        title: Text(item['title']),  
        leading: Checkbox(value: item['done'], onChanged: (value) {}),  
        trailing: const Icon(Icons.delete),  
      ),  
    ),  
  );  
}
```

Section 3

Android Development





Introduction to Android

- **Android is :**
 - Mobile operating system based on Linux kernel
 - Can be considered as a software stack that includes an operating system, middleware, and key applications for mobile and embedded devices
- It includes user Interfaces for touch screens
- Highly customizable for devices / by vendors



Introduction to Android

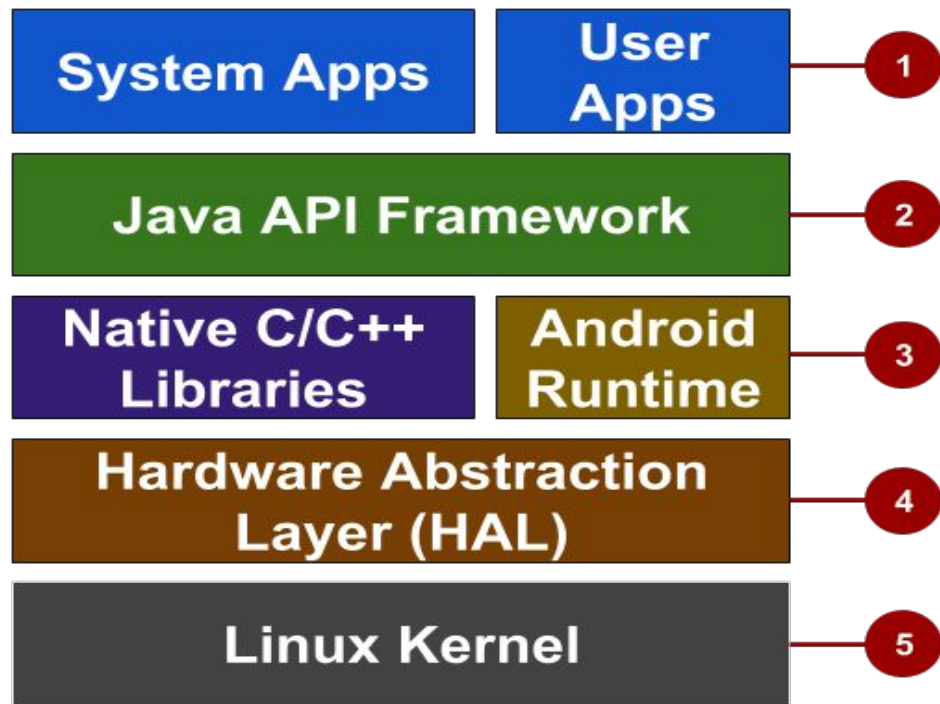
- **Why develop for Android OS:**

- Used on over 80% of all smartphones
- Simple, powerful, and open SDK, no licensing fees, excellent documentation.
- Powers devices such as watches, TVs, and cars
- Billions of users having Android Mobile Phones
- Android runs on different platforms created by numerous manufacturers including : Samsung, LG (Exiting the market), Xiaomi, Motorola, Nokia...

Introduction to Android

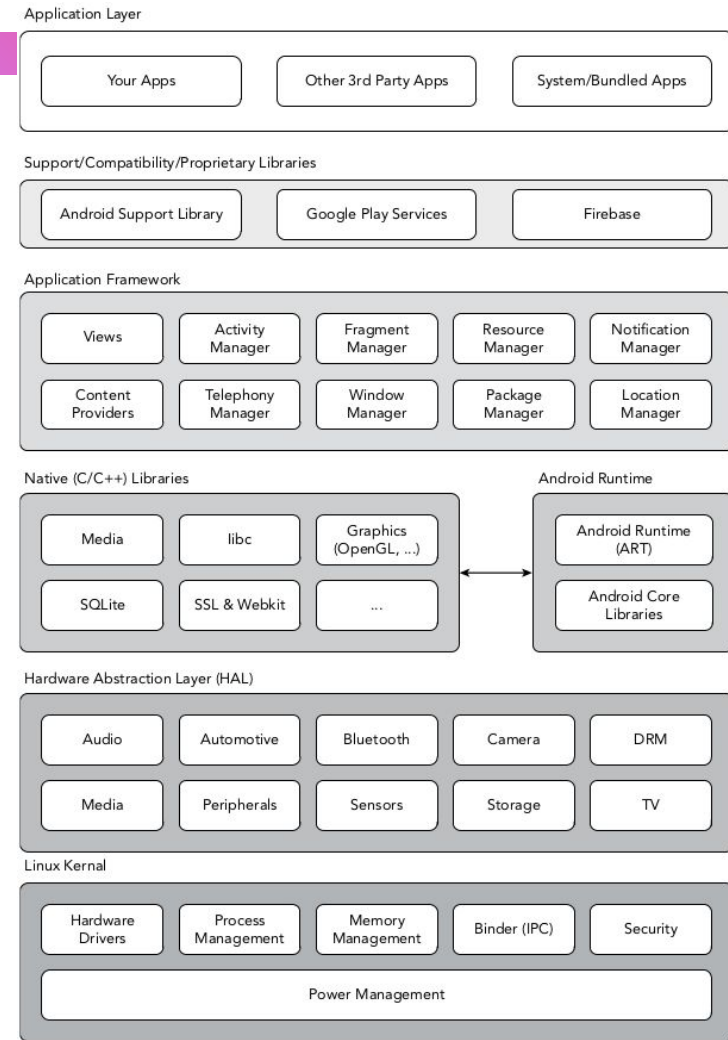
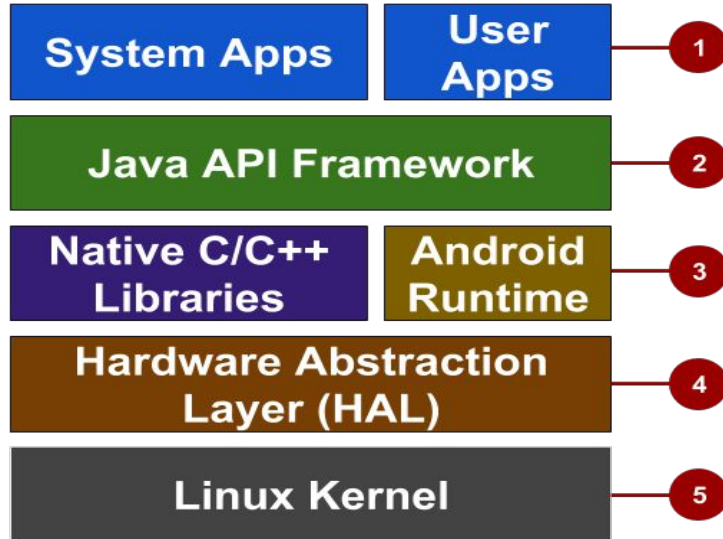
- **Android Operating System Stack**

1. System and user apps
2. Android OS API in Java framework
3. Expose native APIs; run apps
4. Expose device hardware capabilities
5. Linux Kernel

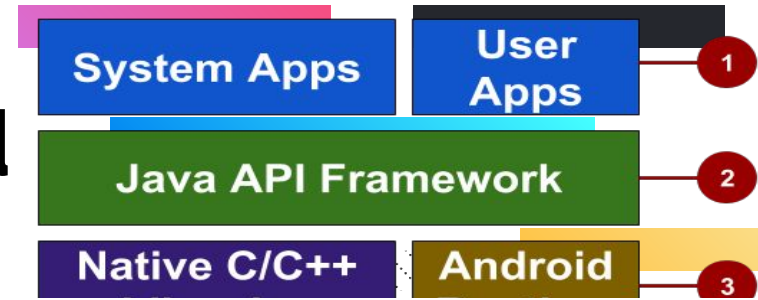


Introduction to Android

- Android Operating System Stack



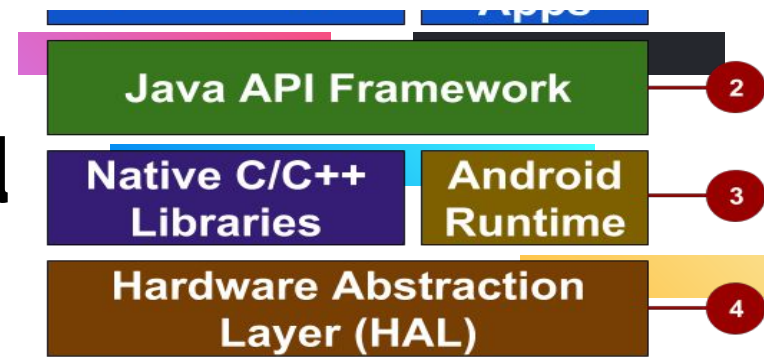
Introduction to Android



- **Android Operating System Stack**

- **User Apps + System Apps** : Apps created by developers along with core system apps for email, SMS messaging, calendars, internet browsing, and contacts.
- **Java API framework** : All features for Android development, such as UI components, resource management, and lifecycle management, are available through application programming interfaces (APIs).

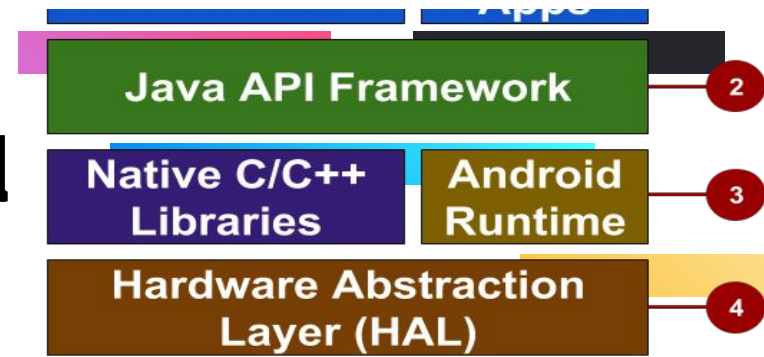
Introduction to Android



- **Android Operating System Stack**

- **Native C/C++ Libraries** : They include various C/C++ core libraries such as libc and SSL, as well as the following:
 - *A media library for playback of audio and video media*
 - *A surface manager to provide display management*
 - *Graphics libraries that include OpenGL for 2D and 3D graphics*
 - *SQLite for native database support*
 - *SSL and WebKit for integrated web browser and Internet security*
- These native libraries are available to apps through the Java API framework.

Introduction to Android

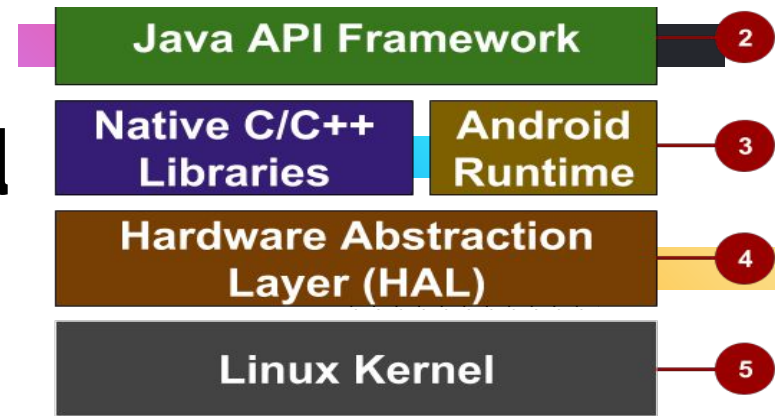


- **Android Operating System Stack**

- **Android Runtime (ART) :**

- It functions the same way as a traditional Java Virtual Machine **BUT**
 - Android uses its own custom run time designed to ensure that **multiple instances** run efficiently on a single device.
 - app runs in its own process, with its own instance of the Android runtime.

Introduction to Android

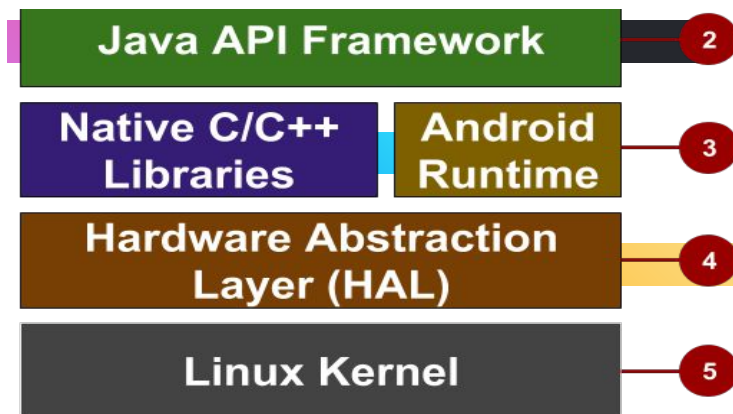


- **Android Operating System Stack**

- **Hardware Abstraction Layer (HAL) :**

- This layer provides standard interfaces that expose device hardware capabilities to the next layer so it can be utilised by the higher-level Application framework.
 - The HAL consists of multiple library modules, each of which implements an interface for a **specific type of hardware component**, such as the camera or Bluetooth module.

Introduction to Android



- **Android Operating System Stack**

- **Linux Kernel :**

- The layers above the Linux kernel rely on the Linux kernel for threading, low-level memory management, power consumption, resources, network, hardware drivers and other underlying core functionality.

Introduction to Android

- Versions :

Android Lollipop	Lemon Meringue Pie	5.0 – 5.0.2	21	November 4, 2014 ^[20]	November 2017	23.37.17 (October 2023)
		5.1 – 5.1.1	22	March 2, 2015 ^[21]	March 2018	
Android Marshmallow	Macadamia Nut Cookie	6.0 – 6.0.1	23	October 2, 2015 ^[22]	August 2018	
Android Nougat	New York Cheesecake	7.0	24	August 22, 2016	August 2019	
		7.1 – 7.1.2	25	October 4, 2016	October 2019	
Android Oreo	Oatmeal Cookie	8.0	26	August 21, 2017	January 2021	
		8.1	27	December 5, 2017	October 2021	
Android Pie	Pistachio Ice Cream ^[23]	9	28	August 6, 2018	January 2022	
Android 10	Quince Tart ^[24]	10	29	September 3, 2019	February 2023	
Android 11	Red Velvet Cake ^[24]	11	30	September 8, 2020	October 2023	
Android 12	Snow Cone	12	31	October 4, 2021		
Android 12L	Snow Cone v2	12.1 ^[a]	32	March 7, 2022		
Android 13	Tiramisu	13	33	August 15, 2022		
Android 14	Upside Down Cake ^[27]	14	34	October 4, 2023		
Android 15	Vanilla Ice Cream ^[28]	15	TBA	Q3 2024	—	—
Legend: Old version Older version, still maintained Latest version Latest preview version Future release						



Introduction to Android

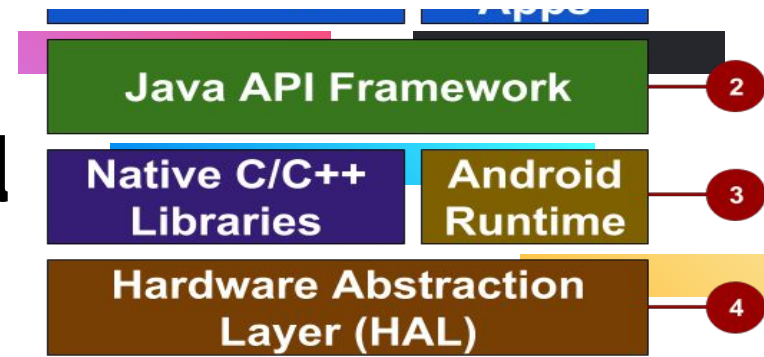
- **Software Development Kit (SDK)** for Android
 - It includes everything you need to start developing, testing, and debugging Android applications:
 - **The Android API Libraries** : provide developer access to the Android stack. These are the same libraries that Google uses to create native Android applications.
 - **Development tools**: including the Android Studio IDE and several other development tools that let you compile and debug your applications to turn Android source code into executable applications.



Introduction to Android

- **Software Development Kit (SDK)** for Android
 - It includes everything you need to start developing, testing, and debugging Android applications:
 - **The Android Virtual Device Manager and Emulator** : The Android Emulator (As a virtual device) runs within an Android Virtual Device (AVD) that simulates a device hardware configuration. Using the Emulator you can see how your applications will look and behave on a real Android device.
 - **Sample code** : SDK includes a selection of sample applications...
 - **Full documentation**

Introduction to Android



- **Android Operating System Stack**

- **Native Development Kit :**

- If the speed and efficiency of C/C++ is required for your application, Android provides a native development kit (NDK).
 - The NDK is designed to enable you to create C++ libraries using the libc and libm libraries, along with native access to OpenGL.

Lecture Demo Apps

- To-Do UI with a ListView

- <https://drive.google.com/file/d/1NcZELqSx3tmKXNO31ej4ykt1ufkkhbZc/view?usp=sharing>



Resources

- <https://www.youtube.com/@flutterdev/videos>
- <https://m3.material.io/>
- <https://codelabs.developers.google.com/codelabs/flutter-codelab-first#0>
- <https://codewithandrea.com/tutorials/>
- <https://www.youtube.com/watch?v=cCMULezpNRQ>
- <https://www.youtube.com/watch?v=L9cPgOTUstA&list=PLjxrf2q8roU23XGwz3Km7sQZFTdBg96iG>
- https://www.youtube.com/watch?v=OBluSrg_Quo&list=PLjxrf2q8roU1fRV4oEc82oorX6OuQkmnl
- https://www.youtube.com/watch?v=fQldtPigGBM&list=PLjxrf2q8roU3N8-rlgg8_jueoNT-85A6_
- <https://gallery.flutter.dev/#/>
- <https://dart-tutorial.com/introduction-and-basics/>